

The Fraud Was Not Random. The Data Showed Exactly Where It Was Hiding.

250,000 transactions. 8 African markets. A 39% rise in fraud that nobody could explain until the data told the story.

I expected fraud to be scattered. A little everywhere. No clear pattern. Just noise.

The data said something completely different.

Crypto Exchange merchants were running at 10.12% fraud rate while the platform average sat at 2.93%. Accounts under 30 days old carried nearly three times the risk of established ones. Transactions where the IP country did not match the transaction country more than doubled the fraud rate. And the whole problem was getting worse. Fraud rate climbed 39% from the first quarter to the last.

None of that was visible before the analysis. This is the story of how I found it, step by step, from a dirty dataset to a 5-page Power BI dashboard with row-level security.

The Business Problem

I created FinLend Solutions, a fictional digital payments company operating across eight African markets: Kenya, Nigeria, Ghana, South Africa, Egypt, Tanzania, Uganda, and Rwanda. The platform moves money three ways. Peer-to-peer transfers. Merchant payments. Mobile wallet top-ups and withdrawals.

I built the dataset with AI and designed realistic fraud patterns into it on purpose. Real fraud data sits behind privacy walls. This is how I work through the actual analytical process without compromising anyone.

The dataset: 250,000 transactions, 15,000 customers, 1,500 merchants, 8 countries. Deliberately dirty. Inconsistent labels. Duplicate rows. Bad amounts. Placeholder emails. Future-dated timestamps. Every issue maps to a real-world cause.

The business problem: fraud losses are climbing but the company cannot see the cause. Four blind spots. Where is fraud happening. Who is most affected. What signals it early. Which segments to fix first.

The goal was not to build a fraud blocker. It was to build visibility.

The Framework I Used

Before writing a single query, I broke the problem down using seven questions: Who, What, Where, When, Why, Which, How.

Who is committing or affected by the fraud? Customers, merchants, fraudsters. What types of fraud are occurring? Chargebacks, identity theft, account takeover. Where do fraudulent transactions happen most? Countries, channels, merchant categories. When do they occur? Late at night, early in account life. Why is fraud increasing, and what is the financial impact? Which payment methods, transaction types, or customer segments have the highest fraud rates? How can we detect fraud earlier or reduce losses?

These seven questions naturally guided the analysis and decided what data I needed, which SQL queries to write, and what dashboards to produce.

Phase 1: Database Setup and Staging

I loaded the data into SQL Server in two layers on purpose.

The first layer is staging tables. Every column is NVARCHAR, which is text. Text accepts anything. A real date, a broken date, a negative amount, a typo. The load always succeeds.

Why not load straight into clean typed tables? Because the data has bad dates, zero amounts, and messy labels. The moment BULK INSERT hits one bad value trying to force it into a DATETIME2 or DECIMAL column, the whole load fails. One bad row kills 250,000 good ones. Staging avoids that entirely.

I loaded the four CSV files with BULK INSERT and verified the row counts: 250,000 transactions, 15,000 customers, 1,500 merchants, 8 countries.

INSERT SCREENSHOT: SQL/3_row_count_verified.png Caption: Row count verification. All four tables loaded.

Phase 2: Data Profiling

Before I cleaned anything, I measured the mess. Six named checks, each targeting a specific data quality issue. This is the step that makes every cleaning decision defensible.

Check 1: Null values. 10,005 missing device IDs, 7,507 missing IP countries, 2,505 missing currency codes. These are optional fields with partial capture. I kept these rows and carried the nulls forward.

INSERT SCREENSHOT: SQL/4_missing_data_profile.png Caption: Null value counts across optional columns.

Check 2: Inconsistent country labels. I ran a GROUP BY on transaction_country and got back 17 distinct values instead of 8. Kenya appeared as Kenya, KENYA, and KE. Uganda appeared as Uganda and " Uganda" with a leading space. Ghana appeared as Ghana and "gh " with a trailing space.

Three kinds of dirt in one column. Case differences (TANZANIA vs Tanzania). Abbreviation substitution (KE, NG, ZA standing in for full names). Hidden whitespace ("

Uganda" with a leading space). The whitespace is the dangerous one. " Uganda" and "Uganda" look identical on screen but SQL treats them as two different countries. That single space would have split Uganda's fraud rate across two rows and silently broken the geography join.

INSERT SCREENSHOT: SQL/5_inconsistent_values.png Caption: 17 distinct country spellings instead of 8. The mess that proves cleaning is not optional.

Check 3: Invalid amounts. 405 transactions with zero or negative amounts. Zero non-numeric values. I filtered them out. A transaction with zero or negative amount is not a real payment.

Check 4: Future-dated timestamps. 60 rows dated in July 2026, past my data window. Every one stamped at exactly 23:59:59. That uniform time is the fingerprint of a system bug. Real transactions scatter across random times.

INSERT SCREENSHOT: SQL/7_future_dates.png Caption: 60 future-dated rows, all at 23:59:59. System bug, not human activity.

Check 5: Duplicate rows. 5,000 rows sitting in duplicate groups. That is 2,500 pairs. I used COUNT(*) OVER (PARTITION BY transaction_id, customer_id, merchant_id, transaction_timestamp, amount) to tag each row with how many copies exist.

Check 6: Placeholder emails. 90 customer accounts carrying fake emails. placeholder@finlend.io (26), test@test.com (18), none (18), na (16), xxx@xxx.com (12). Test accounts left in production.

I did not delete these customers. A customer with a fake email still made real transactions. Deleting them would orphan their transaction history and break every join that references their customer_id. Instead I added an is_valid_email column. 1 for real, 0 for junk. Keep the record, mark the problem.

INSERT SCREENSHOT: SQL/9_placeholder_emails.png Caption: 5 junk email values and their counts. Flagged, not deleted.

Profiling scorecard:

Check	Found
Null values	10,005 device / 7,507 IP / 2,505 currency
Country labels	Up to 3 spellings per country
Bad amounts	405 zero or negative
Future dates	60, all at 23:59:59
Duplicates	5,000 rows (2,500 pairs)
Placeholder emails	90 across 5 junk values

Phase 3: Data Cleaning

I built clean typed tables and moved data across from staging. Dimensions first, fact table last.

dim_geography: 8 rows, already tidy. Real types and a primary key.

dim_customers: Date conversion on `signup_date` plus the `is_valid_email` flag. The flag uses `LIKE '%@%._%'` to check email shape, plus an `IN` list for known junk.

dim_merchants: Date conversion on `onboarding_date`. Cleanest dimension.

fact_transactions: All six issues resolved in a single CTE called "typed." Five things happened inside it:

1. Type conversion with `TRY_CONVERT`. Returns `NULL` instead of crashing on bad values.
2. Country standardization with `CASE`. `UPPER(LTRIM(RTRIM(transaction_country)))` handles case and whitespace before the comparison.
3. Payment method standardization. Same `LTRIM`, `RTRIM`, `LOWER` approach.
4. `NULL` handling with `NULLIF`. Empty strings converted to proper `NULL`s.
5. Deduplication with `ROW_NUMBER() OVER (PARTITION BY the five business fields ORDER BY (SELECT NULL))`.

The outer `WHERE` clause: keep `rn = 1` (drop duplicates), keep rows where the date parsed, keep rows within the data window, keep rows where amount is greater than zero.

Post-clean validation confirmed: 247,040 clean rows, 8 distinct countries, 5 payment methods, 0 remaining duplicates, 0 orphaned customer foreign keys, 0 orphaned merchant foreign keys.

INSERT SCREENSHOT: SQL/11_post_clean_validation.png Caption: All six validation checks passing. The clean layer is proven.

Phase 4: Analysis

I organized the analysis around four groups of business questions. Here are the findings.

Finding 1: Fraud rate rose 39% across the year

From 2.39% in the first quarter to 3.32% in the last. Total fraud value: \$917,576 on \$23.87M in transactions.

I used a CTE to collapse the table to one row per month, then applied window functions for a running total and a 3-month rolling average.

This is the number that would open a board-level risk meeting. Everything else in the analysis supports it.

INSERT SCREENSHOT: SQL/13_monthly_trend.png Caption: Monthly fraud trend. The 39% rise from first quarter to last is clear.

Finding 2: Four merchant categories drive most of the risk

Category	Fraud Rate
Crypto Exchange	10.12%
Gift Cards	8.46%
Betting	7.37%
Gaming	6.90%
Platform average	2.93%

These categories convert to untraceable value instantly. Fraudsters target them because stolen money is hard to recover once the transaction completes. At the other end, Groceries (1.44%) and Utilities (1.21%) are the safe core. The spread is 8x.

INSERT SCREENSHOT: SQL/19_category_risk_ranked.png Caption: All categories ranked. Crypto Exchange at 10.12% leads by a wide margin.

Finding 3: New accounts are nearly three times riskier

Tenure band	Fraud Rate
Under 30 days	6.23%
30 to 90 days	3.69%
90 to 365 days	2.24%
Over a year	2.41%

The drop from under 30 days to 30-90 days is dramatic. I calculated tenure using `DATEDIFF(DAY, signup_date, transaction_timestamp)` and bucketed it with `CASE`.

In practice, this means onboarding controls and transaction limit thresholds need to be set based on account age, not applied uniformly.

INSERT SCREENSHOT: SQL/23_tenure_bands.png Caption: Fraud rate drops as account age increases. The first 30 days are the danger zone.

Finding 4: IP country mismatch is the strongest single signal

6.49% for mismatched transactions versus 2.75% for aligned ones. More than double. A real customer in Kenya transacting in Kenya has a Kenyan IP. When the IP says Egypt and the transaction says Kenya, something is wrong.

If FinLend had a real-time fraud engine, this would be the first rule in it.

INSERT SCREENSHOT: SQL/17_ip_mismatch.png Caption: IP mismatch more than doubles the fraud rate. The strongest single signal.

Finding 5: Overnight hours carry double the fraud rate

Transactions between 12AM and 5AM run at 5.59%. Daytime sits between 2.74% and 2.99%. Real customers sleep. Automated fraud systems do not.

INSERT SCREENSHOT: SQL/16_hourly_clustering.png Caption: Overnight hours at 5.59% versus daytime at 2.75%. Real customers sleep.

Finding 6: The geographic story is corridors, not countries

This was the finding I had to be most honest about.

Country-level fraud rates are nearly flat. Ghana at 3.09%. Uganda at 2.83%. A spread of only 0.26 percentage points. It would have been easy to say "Ghana has the highest fraud rate." But a 0.26 percentage point difference is not meaningful.

The real signal is in cross-border corridors. Tanzania to Rwanda at 10.29%. Rwanda to Ghana at 10.09%. Egypt to Ghana at 8.65%. When a customer transacts outside their home

country, the risk changes entirely.

INSERT SCREENSHOT: SQL/22_cross_border_corridors1.png Caption: Top corridors by fraud rate. Tanzania-Rwanda leads at 10.29%.

Finding 7: One customer stands out across multiple dimensions

CUST112663. 82 total transactions. 37 confirmed fraud. 45.12% personal fraud rate. 4 rapid bursts within 24 hours. This customer appeared in both the top fraud value list and the velocity detection query independently.

INSERT SCREENSHOT: SQL/25_velocity_rings.png Caption: Velocity detection. CUST112663 flagged across two independent queries.

Phase 5: Optimization

I measured the baseline cost: 4,368 logical reads, 213ms elapsed time. The fact table was a heap with no indexes. Every query scanned it top to bottom.

I added five indexes. A clustered index on a surrogate key. Nonclustered indexes on customer_id and merchant_id. A covering index on transaction_timestamp with is_fraud, amount, and merchant_id as included columns. A nonclustered index on transaction_country.

After indexing: 658 logical reads, 116ms elapsed time. An 85% reduction in logical reads. The covering index did the heavy lifting. Instead of scanning the whole table, SQL Server seeks directly to the rows it needs.

Metric	Before	After	Change
Logical reads	4,368	658	85% reduction
CPU time	172ms	94ms	45% faster
Elapsed time	213ms	116ms	46% faster

I also rewrote a correlated subquery as a window function, replacing 247,040 sub-executions with a single pass.

INSERT SCREENSHOT: SQL/28_after_optimization_reads.png Caption: Logical reads after indexing. 658 versus the original 4,368.

Phase 6: Power BI Dashboard

I built a 5-page dashboard on a star schema data model. One fact table, four dimensions: customers, merchants, geography, and a date table. All relationships one-to-many, single direction, from dimension to fact.

The DAX Measures

Every number on the dashboard comes from a DAX measure. Here are the core ones:

Fraud Rate:

```
Fraud Rate Pct =  
VAR _fraud = CALCULATE(COUNTROWS(fact_transactions), fact_transactions[is_fraud] =  
TRUE)  
VAR _total = COUNTROWS(fact_transactions)  
RETURN  
DIVIDE(_fraud, _total, 0)
```

One thing I learned building these measures: BIT columns from SQL Server load into Power BI as TRUE/FALSE, not 1/0. Every DAX filter that used `is_fraud = 1` failed. Changing it to `is_fraud = TRUE` fixed it. This caused three separate measure failures before the pattern was clear.

Fraud Value:

```
Fraud Value = CALCULATE(SUM(fact_transactions[amount_in_usd]),  
fact_transactions[is_fraud] = TRUE)
```

Chargeback Value:

```
Chargeback Value = CALCULATE(SUM(fact_transactions[amount_in_usd]),  
fact_transactions[is_chargeback] = TRUE)
```

Page 1: Executive Overview

Four KPI cards. Fraud rate 2.93%. Total value \$23.87M. Fraud value \$917.58K. Chargeback value \$591.16K. A fraud rate trend line showing the rise from 2.11% to 3.36%. Fraud rate by channel. Fraud value by month.

This page was designed so a non-technical risk lead can open it on Monday morning and know exactly where the problem stands without asking anyone.

INSERT SCREENSHOT: PowerBI/Executive_Overview.png Caption: Page 1. The rising trend line is the headline visual.

Page 2: Fraud Trends

Fraud rate by payment method. Fraud rate by time band with overnight at 5.59%. IP mismatch versus aligned (6.49% vs 2.75%). Fraud type breakdown showing even distribution across all six types.

INSERT SCREENSHOT: PowerBI/Fraud_Trends.png Caption: Page 2. IP mismatch at 6.49% is the strongest single signal on the dashboard.

Page 3: Merchant Risk

Fraud rate by merchant category with Crypto Exchange at 10.12% at the top. A merchant table with conditional formatting on fraud rate. A scatter plot showing volume versus fraud rate, sized by fraud value. The dangerous merchants appear as large bubbles in the top-right quadrant.

INSERT SCREENSHOT: PowerBI/Merchant_Risk.png Caption: Page 3. Crypto Exchange dominates. The scatter plot shows the danger zone.

Page 4: Geographic Analysis

Fraud rate by country showing the flat spread. The cross-border corridor table. The country bars prove the point. The story is not which country is worse. It is which corridors are vulnerable.

INSERT SCREENSHOT: PowerBI/Geographic_Analysis.png Caption: Page 4. Country rates are flat. The corridors tell the real story.

Page 5: Customer Profiles

Fraud rate by account tenure. Fraud rate by KYC status. Top customers by fraud value. Velocity detection flagging 13 customers with 10 or more fraud transactions.

INSERT SCREENSHOT: PowerBI/Customer_Profiles.png Caption: Page 5. Under 30 days at 6.23% is the clearest customer-level signal.

Row-Level Security

I set up four regional roles: East Africa, West Africa, Southern Africa, North Africa. Each role filters dim_geography by region. When viewing as East Africa, total value drops from \$23.87M to \$9.04M and the dashboard shows only Kenya, Tanzania, Uganda, and Rwanda.

INSERT SCREENSHOT: PowerBI/rls_test1.png Caption: RLS in action. Viewing as East Africa. Every visual filters automatically.

Lessons Learned

1. Always stage data as text before typing it. One bad value in a DATETIME2 column crashes the entire BULK INSERT. Text columns accept everything.
2. Profile before you clean. Without measuring the dirt first, there is no evidence behind the cleaning decisions.
3. BIT columns from SQL Server behave as TRUE/FALSE in Power BI, not 1/0. This caused three separate measure failures before the pattern was clear.

4. The strongest geographic signal is often not at the country level. Country rates were flat. The real story was cross-border corridors running at 10%+.
 5. Flag dirty records instead of deleting them. A customer with a fake email still made real transactions. Deleting the customer breaks every join downstream.
 6. Optimization is a before-and-after story. Capture the baseline before adding indexes. Without the before number, the improvement has no proof.
-

What I Would Do Differently

1. Add a numeric surrogate key to the geography dimension instead of joining on a text label. Text joins depend on perfect standardization. A numeric key is immune to spelling issues.
 2. Build the date table from the start. Retrofitting it caused a relationship issue with DATETIME2 timestamps.
 3. Explore a columnstore index on the fact table for aggregation-heavy dashboard queries.
 4. Add a proper currency conversion step. The amounts are USD equivalent by design, but a real platform would need explicit conversion with recorded exchange rates.
-

Recommendations

1. Tighten controls on Crypto Exchange, Gift Cards, Betting, and Gaming merchants. These four categories run three to eight times the platform average.
 2. Enforce KYC completion and lower limits during the first 30 days. New accounts carry a 6.23% fraud rate.
 3. Add step-up verification for IP mismatch and overnight transactions. These are the two strongest behavioral signals.
 4. Apply corridor-specific velocity rules for cross-border transactions. Tanzania-Rwanda, Rwanda-Ghana, and Egypt-Ghana all exceed 8%.
 5. Investigate CUST112663 and similar velocity-burst accounts immediately.
-

The Full Project

GitHub (code, data, screenshots): github.com/Abijahkabi/finlend-fraud-analysis

LinkedIn: [linkedin.com/in/abijahkabi](https://www.linkedin.com/in/abijahkabi)

Portfolio: abijahkabi.github.io

Built by Abijah Kabi. Data Analyst. Nairobi, Kenya.